0295. Find Median from Data Stream

gory: Heap / Priority Queue | LeetCode: https://leetcode.com/problems/find-medi

Problem

- Support addNum(x) and findMedian() over a growing stream.

Intuition

- Split the numbers into a lower half (max-heap) and an upper half (min-heap).
- Keep sizes balanced (lower may hold one extra). The median is lower's top, or the average of both tops.

Brute Force

- Keep a sorted list with insertion: O(n) per add.

Optimal Approach

- AddNum: push to lower; move lower's top to upper (keeps the order invariant).
- If upper is bigger than lower, move upper's top back to lower.
- FindMedian: sizes equal → average of tops; else lower.peek().

Worked Example

```
add 1: lower {1}           median 1
add 2: lower {1} upper {2} median 1.5
add 3: lower {2,1} upper {3}  median 2
```

Java Solution

```java
public class MedianFinder {
    private final PriorityQueue<Integer> lower = new PriorityQueue<>(Collections.reverseOrder());
    private final PriorityQueue<Integer> upper = new PriorityQueue<>();

    public void addNum(int num) {
        lower.offer(num);
        upper.offer(lower.poll());
        if (upper.size() > lower.size()) lower.offer(upper.poll());
    }

    public double findMedian() {
        if (lower.size() > upper.size()) return lower.peek();
        return ((double) lower.peek() + upper.peek()) / 2.0;
    }
}
```

Complexity

- AddNum O(log n), findMedian O(1). Space O(n).

Pitfalls

- Average as double; sum of two ints can overflow, so cast before adding.